

MySQL Capstone Project

Online Shopping System

Submitted By: Varad Kompelli

Date: 28/07/2026

Institute: SevenMentor Pvt. Ltd.

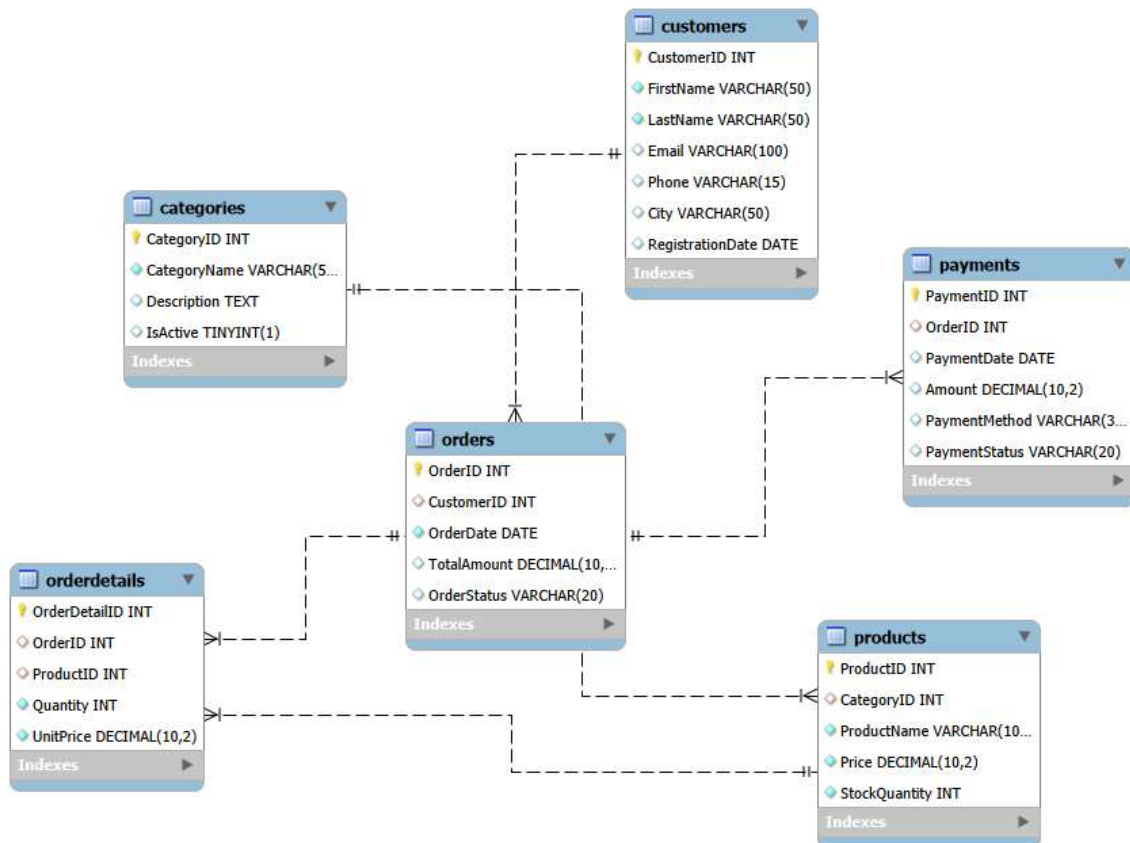
1. Objective

This project demonstrates a practical understanding of MySQL through the design and implementation of an Online Shopping System database. It covers database design, DDL, DML, Joins, Grouping, and Advanced SQL querying for real-world business reports.

2. Database Design & ER Diagram

The database consists of 6 core tables properly linked using Primary and Foreign keys:

- Customers (CustomerID, FirstName, LastName, Email, Phone, City, RegistrationDate)
- Categories (CategoryID, CategoryName, Description, IsActive)
- Products (ProductID, CategoryID, ProductName, Price, StockQuantity)
- Orders (OrderID, CustomerID, OrderDate, TotalAmount, OrderStatus)
- OrderDetails (OrderDetailID, OrderID, ProductID, Quantity, UnitPrice)
- Payments (PaymentID, OrderID, PaymentDate, Amount, PaymentMethod, PaymentStatus)



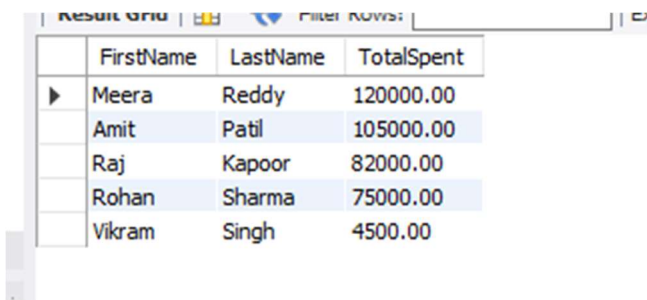
3. Business Reports (Queries & Output)

Report 1: Top 5 customers by revenue

SQL Query:

```
SELECT c.FirstName, c.LastName, SUM(o.TotalAmount) AS TotalSpent
FROM Customers c
INNER JOIN Orders o ON c.CustomerID = o.CustomerID
WHERE o.OrderStatus != 'Cancelled'
GROUP BY c.CustomerID
ORDER BY TotalSpent DESC
LIMIT 5;
```

Output:



The screenshot shows a database query result grid with the following data:

	FirstName	LastName	TotalSpent
▶	Meera	Reddy	120000.00
	Amit	Patil	105000.00
	Raj	Kapoor	82000.00
	Rohan	Sharma	75000.00
	Vikram	Singh	4500.00

Report 2: Highest revenue generating product

SQL Query:

```
SELECT p.ProductName, SUM(od.Quantity * od.UnitPrice) AS TotalRevenue
FROM Products p
INNER JOIN OrderDetails od ON p.ProductID = od.ProductID
GROUP BY p.ProductID
ORDER BY TotalRevenue DESC
LIMIT 1;
```

Output:

	ProductName	TotalRevenue
►	Dell XPS 13	120000.00

Report 3: Monthly Sales Trend (2026)

SQL Query:

```
SELECT MONTHNAME(OrderDate) AS Month, SUM(TotalAmount) AS MonthlyRevenue
FROM Orders
WHERE YEAR(OrderDate) = 2026
GROUP BY MONTH(OrderDate), MONTHNAME(OrderDate)
ORDER BY MONTH(OrderDate);
```

Output:

	Month	MonthlyRevenue
▶	July	414100.00

Report 4: Most ordered product by quantity

SQL Query:

```
SELECT p.ProductName, SUM(od.Quantity) AS TotalUnitsSold
FROM Products p
INNER JOIN OrderDetails od ON p.ProductID = od.ProductID
GROUP BY p.ProductID
ORDER BY TotalUnitsSold DESC
LIMIT 5;
```

Output:

	ProductName	TotalUnitsSold
►	Organic Green Tea	3
	Cotton Polo T-Shirt	2
	Floral Summer Dress	2
	Samsung Galaxy S23	1
	Denim Jeans Men	1

Report 5: Average Order Value

SQL Query:

```
SELECT ROUND(AVG(TotalAmount), 2) AS AverageOrderValue  
FROM Orders;
```

Output:

	AverageOrderValue
▶	20705.00

Report 6: Category-wise sales exceeding ₹5000

SQL Query:

```
SELECT c.CategoryName, SUM(od.Quantity * od.UnitPrice) AS CategoryRevenue
FROM Categories c
INNER JOIN Products p ON c.CategoryID = p.CategoryID
INNER JOIN OrderDetails od ON p.ProductID = od.ProductID
GROUP BY c.CategoryName
HAVING CategoryRevenue > 5000
ORDER BY CategoryRevenue DESC;
```

Output:

	CategoryName	CategoryRevenue
▶	Laptops	225000.00
	Smartphones	157000.00
	Footwear	7700.00

Report 7: Customer purchase history (All Customers)

SQL Query:

```
SELECT c.FirstName, c.LastName, COUNT(o.OrderID) AS TotalOrders,
IFNULL(SUM(o.TotalAmount), 0) AS TotalSpent
FROM Customers c
LEFT JOIN Orders o ON c.CustomerID = o.CustomerID
GROUP BY c.CustomerID
ORDER BY TotalOrders DESC;
```

Output:

	FirstName	LastName	TotalOrders	TotalSpent
►	Rohan	Sharma	1	75000.00
	Priya	Desai	1	2000.00
	Amit	Patil	1	105000.00
	Neha	Joshi	1	1200.00
	Vikram	Singh	1	4500.00
	Anjali	Mehta	1	3500.00
	Rahul	Verma	1	3200.00
	Sneha	Kulkarni	1	400.00
	Karan	Malhotra	1	1800.00
	Pooja	Rao	1	350.00
	Siddharth	Nair	1	600.00
	Kavita	Iyer	1	750.00
	Manish	Tiwari	1	2200.00
	Divya	Bhat	1	1800.00
	Aditya	Chavan	1	3000.00
	Shruti	Pandey	1	4500.00
	Raj	Kapoor	1	82000.00
	Meera	Reddy	1	120000.00
	Tarun	Bose	1	800.00
	Aarti	Yadav	1	1500.00

Report 8: Products that have never been ordered

SQL Query:

```
SELECT p.ProductName, p.StockQuantity
FROM OrderDetails od
RIGHT JOIN Products p ON od.ProductID = p.ProductID
WHERE od.OrderDetailID IS NULL;
```

Output:

	ProductName	StockQuantity
►	Philips Mixer Grinder	25

Report 9: Pending payments report with customer details

SQL Query:

```
SELECT c.FirstName, c.LastName, c.Phone, o.OrderID, p.Amount,  
p.PaymentMethod  
FROM Customers c  
INNER JOIN Orders o ON c.CustomerID = o.CustomerID  
INNER JOIN Payments p ON o.OrderID = p.OrderID  
WHERE p.PaymentStatus = 'Pending';
```

Output:

	FirstName	LastName	Phone	OrderID	Amount	PaymentMethod
▶	Vikram	Singh	9876543214	5	4500.00	Net Banking
	Karan	Malhotra	9876543218	9	1800.00	UPI
	Divya	Bhat	9876543223	14	1800.00	UPI
	Meera	Reddy	9876543227	18	120000.00	Credit Card

Report 10: Overall Summary Dashboard

SQL Query:

```
CREATE VIEW StoreDashboard AS
SELECT
    (SELECT COUNT(*) FROM Customers) AS TotalCustomers,
    (SELECT COUNT(*) FROM Orders) AS TotalOrders,
    (SELECT SUM(TotalAmount) FROM Orders WHERE OrderStatus = 'Delivered')
AS TotalRevenue,
    (SELECT MIN(TotalAmount) FROM Orders) AS MinimumOrderValue,
    (SELECT MAX(TotalAmount) FROM Orders) AS MaximumOrderValue;

SELECT * FROM StoreDashboard;
```

Output:

	TotalCustomers	TotalOrders	TotalRevenue	MinimumOrderValue	MaximumOrderValue
▶	20	20	276500.00	350.00	120000.00

4. Conclusion

In conclusion, this Capstone Project successfully demonstrates the end-to-end design and implementation of a relational database for an Online Shopping System. By structuring normalized tables with appropriate primary and foreign keys, the project ensures data integrity and a scalable architecture. Furthermore, the diverse set of SQL queries—ranging from basic data manipulation to complex joins, aggregate functions, and views—highlights the ability to extract meaningful business intelligence from raw data. Ultimately, this project serves as a comprehensive showcase of the practical MySQL skills required for real-world database management and data analysis.